

# SHARP: A Smart Hostel Administration and Resource Platform

Digital Workflows for Leave, Mess, Allotment, Maintenance and Laundry Services

Ali Ekram

CSED, Roll No. 1024160138

Thapar Institute of Engineering and Technology  
Patiala, India

Riya Gupta

CSED, Roll No. 1024160122

Thapar Institute of Engineering and Technology  
Patiala, India

Vaibhav Kansal

CSED, Roll No. 1024160097

Thapar Institute of Engineering and Technology  
Patiala, India

Ayush Kamboj

CSED, Roll No. 1024160118

Thapar Institute of Engineering and Technology  
Patiala, India

**Abstract**—Hostel administration still depends on handwritten applications, physical registers and loosely tracked complaints, which produces delays, broken room clusters, unresolved maintenance and lost laundry. SHARP is a centralised web platform unifying five such processes—leave and visitor passes, mess feedback, room allotment, maintenance complaints and laundry—behind role-based workflows for students, parents, wardens, caretakers, mess staff and laundry staff. The goal is not digitisation alone but traceability and accountability: complaints close only on student verification, room reservations expire automatically, laundry batches are tracked by reusable QR identity, and mess feedback stays anonymous to mess staff while remaining auditable.

**Index Terms**—hostel management, workflow automation, role-based access control, SLA escalation, QR tracking, web application

## I. INTRODUCTION

Several routine hostel processes are partially manual or lack tracking, so requests move through paper, phone calls and registers without an auditable trail: leave approvals are delayed, complaint status is unknown, clusters break under staggered fee payment, and laundry is lost without recourse. SHARP gives every participant one platform for the services their role covers, and each module adds a mechanism the manual process lacks—a verification step, an expiry rule or an escalation timer—rather than reproducing paperwork on a screen.

## II. PROBLEM STATEMENT

Five pain points motivate the system. *Leave and passes*: handwritten applications and manual parent contact create paperwork and delay. *Mess feedback*: register-based collection discourages honesty because identity is visible. *Allotment*: when cluster members pay at different times, a partly reserved room may be taken by others, breaking clusters. *Maintenance*: complaints stay unresolved for days, progress is untrackable and preferred slots go unhonoured. *Laundry*: manual slips

leave no record of batches, so losses stay invisible. Each is a traceability gap.

## III. PROPOSED SYSTEM

SHARP comprises five modules. *Visitor and leave management* handles day-out, regular and emergency requests carrying start and end time, destination, reason and emergency contact, routed as Student → Parent verification → Warden approval → digital pass with a unique ID shown at exit. *Mess management* takes categorised feedback on quality, hygiene, quantity, menu, timing and service plus special meal requests such as khichdi during illness or Jain diets; feedback is anonymous to mess staff while internal records deter abuse, and administrators see aggregates such as a weekly rise in food-quality complaints.

*Allotment* adds a temporary reservation: a cluster leader holds a room while members complete payment, and the room advances through Available → Temporarily Reserved → Partially Confirmed → Fully Confirmed → Allotted. The reservation expires after a set window, say 48 hours, returning the room to the pool—which preserves clusters without weakening the fee requirement.

*Complaint management* issues an ID per maintenance report and runs the lifecycle of Fig. 1; what distinguishes it from a status field is student verification before closure and SLA escalation to the warden. *Laundry management* replaces permanent markings on clothing with a reusable QR-coded bag bound to the student's account: request → scan → items received → washing → ready → collection → confirmation. Counts are recorded by category on receipt, the code is rescanned on return, and reports of missing or damaged items become statistics that expose recurring failures.

## IV. SYSTEM ARCHITECTURE

SHARP uses a layered architecture (Fig. 2): a role-specific interface layer, an application and REST API layer carrying

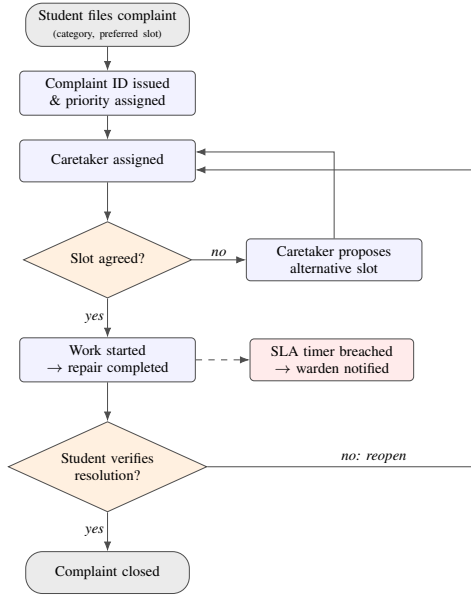


Fig. 1. Maintenance complaint lifecycle. Closure requires student verification, and a negative answer reopens the ticket against the same ID; an SLA timer escalates independently.

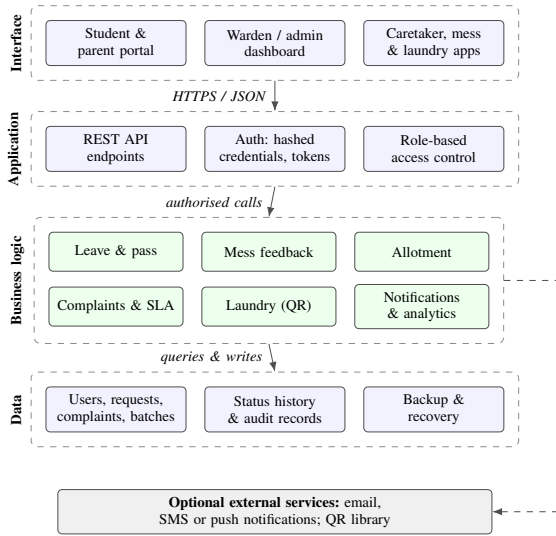


Fig. 2. Layered architecture. Each block names a role rather than a product, so the stack can be fixed later; external services stay optional.

authentication and authorisation, a business logic layer holding the five modules, a persistence layer, and optional external services. Access control is cross-cutting—every sensitive endpoint checks both identity and permission, so a student cannot reach administrative dashboards, parent records or the identity behind anonymous feedback—and modules stay separable, so one can change without redesigning the application.

#### A. Technology and Language Flexibility

The layers are specified by responsibility, not by product, so the implementation language and framework will be decided at the time of building, once the team has weighed hosting,

library maturity and its own familiarity. A JavaScript stack—React with Tailwind CSS, Node.js with Express, MongoDB, token authentication and a QR library—is a proven candidate and is used for costing, but any stack offering a REST-capable framework, a managed database, secure token authentication and a test framework satisfies the design. Deferring the choice costs nothing: no workflow rule, role definition or metric depends on a particular language.

### V. REQUIREMENTS

Students file leave requests, track approvals, hold passes, submit anonymous feedback and meal requests, raise and reopen complaints with preferred slots, and track laundry. Parents act on leave notifications; wardens approve leave, manage rooms and clusters, assign caretakers and view analytics; caretakers accept assignments, propose slots and update status; mess and laundry staff work only within their own queues. Non-functionally, operations should complete within seconds at hostel-scale load, passwords must be hashed, parent approval tokens must resist unauthorised use, the service must stay available outside administrative hours, and the schema should admit further hostels without redesign.

### VI. FEASIBILITY AND COST

The system needs no special hardware, since QR scanning uses phone cameras, and the toolchain is free or open source: the prototype costs roughly INR 0–3,500, dominated by optional hosting, domain, SMS and QR cards. It is operationally feasible because it replaces workflows students already understand. Delivery is phased—authentication and roles, then leave, complaints, mess, allotment, laundry, and finally notifications and deployment—so a working minimum viable product exists even if later phases are unfinished.

### VII. RISKS AND SUCCESS METRICS

Risks and mitigations are: unauthorised access, met by role-based authentication; forged parent approval, met by secure tokens plus warden verification; abuse of anonymous feedback, met by rate limits and audit records; premature loss of a reservation, met by expiry rules and transactional validation; lost laundry, met by batch tracking and collection confirmation; and server failure, met by backup and recovery. Success is measured by the fall in paper applications, average leave approval and complaint resolution times, share of complaints closed within SLA, reopen count, laundry complaints per hundred batches and reservation conflicts avoided—all computable from the platform’s own records.

### VIII. CONCLUSION

SHARP turns five disconnected hostel processes into one auditable platform, adding verification, expiry and escalation mechanisms rather than merely replacing paper. It is feasible technically, economically and operationally, with a mobile application, predictive maintenance and multi-campus support as future scope.